

Atividade de LLP

1 - Estrutura de dados é uma forma de organizar informações EX: Um vetor pode guardar as notas dos alunos

2 - É uma estrutura que armazena vários valores do mesmo tipo em diferentes índices

3 - a) 4

b) 16

c) índice 4

d) 5 elementos

4 - O vetor usa posição por índice a lista encadeada usa nós ligados por ponteiros

5 - É uma parte da lista que contém um valor e um ponteiro para o próximo nó

6 - Aponta para o próximo nó da lista

7 - Porque indica o fim da lista

8- Vetor Acesso rápido e fácil utilização

Lista: tamanho flexível e inserção / remoção facilitada

```
9- int v[10];  
for (int i=0; i<10; i++)  
    cin >> v[i];
```

```
for (int i=0; i<10; i++)  
    cout << v[i] << " ";
```

```
10- int v[10], maior, menor;  
for (int i=0; i<10; i++)  
    cin >> v[i];
```

```
maior = menor = v[0];
```

```
for (int i=1; i<10; i++) {  
    if (v[i] > maior)  
        maior = v[i];  
    if (v[i] < menor) menor = v[i];  
    cout << maior << " " << menor;
```

```
11- float n[5], soma=0;  
for (int i=0; i<5; i++) {  
    cin >> n[i];  
    soma += n[i];  
}  
cout << soma/5;
```

```

12 - int V[10], pares=0, IMPARES=0;
for (int i=0; i<10; i++) {
    cin >> V[i];
    if (V[i]/2 == 0)
        pares++;
    else
        IMPARES++;
}
cout << pares << " " << IMPARES;

```

```

13 - int V[10], x;
    bool achou = false;
    for (int i=0; i<10; i++)
        cin >> V[i];
    for (int i=0; i<10; i++)
        if (V[i] == x) {
            cout << i;
            achou = true;
        }
    if (!achou)
        cout << "Valor não encontrado";

```

14 - a) 10

b) 45

c) 30

d) 45

e) NULL

15 inicial: 10 → 20 → 30 → Null

Com 5: 5 → 10 → 20 → 30 → Null

Com 40: 5 → 10 → 20 → 30 → 40 → Null

16 - a) 10 → 20 → 30 → 40 → Null

b) 10 → 20 → 40 → Null

c) 10 → 20 → Null

17

```
void insert (int valor, No* inicio) {
```

```
    No* novo = new No;
```

```
    novo->valor = valor;
```

```
    novo->proximo = inicio;
```

```
    inicio = novo;
```

18 void listar (No* inicio) {

```
    while (inicio != nullpt) {
```

```
        cout << inicio->valor << " ";
```

```
        inicio = inicio->proximo;
```

```
    }
```

19 - bool busca (No* inicio, int valor) {

```
    while (inicio != nullpt) {
```

```
        if (inicio->valor == valor)
```

```
            return true;
```

```
        inicio = inicio->proximo;
```

```
    }
```

```
    return false;
```

```

20 float n[10], soma = 0
    for (int i=0; i<10; i++) {
        cin >> n[i];
        soma += n[i];
    }
    cout << "Média" << soma/10;

```

```

21 int op;

```

```

do {

```

```

    cout << "\n1 - Inserir,"
    cout << "\n2 - Listar";
    cout << "\n3 - Buscar";
    cout << "\n5 - Contar";
    cout << "\n0 - Sair\n";

```

```

    cin >> op;
} while (op != 0);

```

2) - Veta acesso rápido, mas inserções e remoções são mais difíceis

Lista: facilita inserções e remoções, mas o acesso é mais lento

Mais adequada: lista encadeada quando os registros constantemente